Title

Seat Reservation System Using Multithreading and Synchronization in Java

Objective

To write a Java program to implement a seat reservation system using multiple threads and synchronization, allowing only one person thread at a time to access the `reserve()` method.

System Requirements

Hardware Requirements

- Computer/Laptop
- Minimum 4 GB RAM
- Keyboard and Mouse

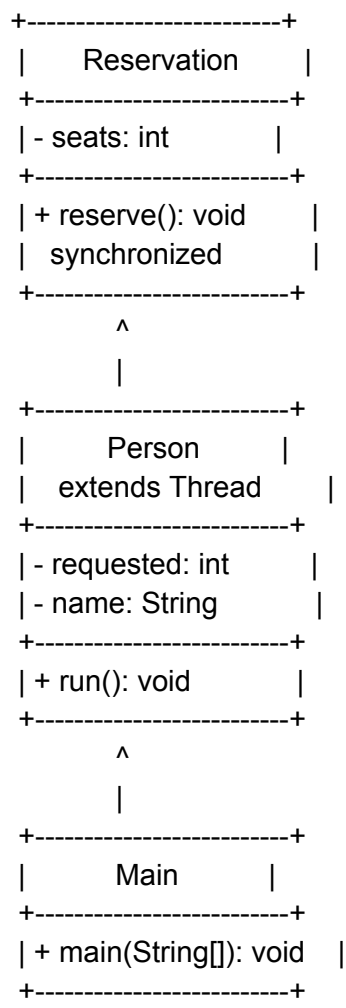
Software Requirements

- Java JDK
- VS Code / Eclipse / IntelliJ IDEA
- Windows/Linux operating system

Algorithm / Logic Description

1. Start the program.
2. Create a `Reservation` class with an initial seat count of 10.
3. Define a synchronized `reserve()` method to book seats.
4. Display the person's name, available seats, and requested seats.
5. Check whether the requested seats are available.
6. If seats are available, reserve them and update the available seat count.
7. Otherwise, display a message indicating that seats are unavailable.
8. Create a `Person` class extending the `Thread` class.
9. Override the `run()` method to call the `reserve()` method.
10. Create three person threads requesting 5, 2, and 4 seats, respectively.
11. Start the threads in sequence using `start()` and `join()` to obtain the expected output order.
12. Stop the program.

UML Class Diagram



Source Code

```
class Reservation {

    int seats = 10;



    synchronized void reserve(int requested, String name) {

        System.out.println(name + " entered.");

        System.out.println("Available seats: " + seats

                + " Requested seats: " + requested);
```

```
        if (seats >= requested) {
            System.out.println("Seat Available. Reserve now :-)");
            seats -= requested;
            System.out.println(requested + " seats reserved.");
        } else {
            System.out.println("Requested seats not available :-)");
        }

        System.out.println(name + " leaving.");
        System.out.println("-----------------------------------------");
    }
}

class Person extends Thread {
    Reservation r;
    int requested;
    String name;

    Person(Reservation r, int requested, String name) {
        this.r = r;
        this.requested = requested;
        this.name = name;
    }
```

```
    public void run() {
        r.reserve(requested, name);
    }
}

public class Main {
    public static void main(String[] args) {
        Reservation r = new Reservation();

        Person p1 = new Person(r, 5, "Person-1");
        Person p2 = new Person(r, 2, "Person-2");
        Person p3 = new Person(r, 4, "Person-3");

        p1.start();
        try {
            p1.join();
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }

        p2.start();
        try {
            p2.join();
```

```
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }

        p3.start();
    }
}
```

Compilation and Execution Commands

Compile:

javac Main.java

Execute:

java Main

Output Screen

```
Person-1 entered.
Available seats: 10 Requested seats: 5
Seat Available. Reserve now :-)
5 seats reserved.
Person-1 leaving.
---------------------------------------------
Person-2 entered.
Available seats: 5 Requested seats: 2
Seat Available. Reserve now :-)
2 seats reserved.
Person-2 leaving.
---------------------------------------------
Person-3 entered.
Available seats: 3 Requested seats: 4
Requested seats not available :-)
```

Person-3 leaving.

```
PS C:\Users\navee\OneDrive\Desktop\javaprograms\labprograms\Week12>  & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '-agentlib:jdwp=transport=dt_socket,server=n,suspend=y,address=localhost:53913' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\navee\AppData\Roaming\Code\User\workspaceStorage\ea10231e2c23d1361f17412c325b029f\redhat.java\jdt_ws\Week12_2b8867f3\bin' 'Week12'
Person-1 entered.
Available seats: 10 Requested seats: 5
Seat Available. Reserve now :-)
5 seats reserved.
Person-1 leaving.
--------------------------------------------
Person-2 entered.
Available seats: 5 Requested seats: 2
Seat Available. Reserve now :-)
2 seats reserved.
Person-2 leaving.
--------------------------------------------
Person-3 entered.
Available seats: 3 Requested seats: 4
Requested seats not available :-)
Person-3 leaving.
--------------------------------------------
```

Result

The Java program was successfully implemented using multithreading and synchronization to ensure that only one person thread accesses the reservation method at a time.